Fine-Grained Complexity

An Introduction

Antonis Antonopoulos

9/11/2018

CoReLab, NTUA

Table of contents

Introduction

Introduction

- The time hierarchy Theorem states that there are problems solved in $T(n)$, for a constructible function T, but not in $\mathcal{O}\left(T^{1-\varepsilon}(n)\right)$, for $\varepsilon > 0$.
- All known b^n-time kSAT algorithms have: $\lim_{k\to\infty} b = 2$ (all the results are of the form $\mathcal{O}^*\left(2^{n-\frac{cn}{k}}\right)$).
- **NP**-hardness is a useful tool for polynomial time hardness, but it doesn't capture non-polynomial times. For example, an $\mathcal{O}^*\left(2^{\sqrt{n}}\right)$ algorithm for kSAT would be a great advancement. It would be difficult the **P** vs **NP** conjecture to be strong enough to prove exponential lower bounds for kSAT.
- The optimality conjecture of the exponential-time kSAT algorithms is formalized as the Strong Exponential Time Hypothesis (*Impagliazzo, Paturi, 2001*).

Introduction

Exponential Time Hypothesis [ETH]

There exists an $\varepsilon > 0$ such that 3SAT requires $2^{\varepsilon n}$ time.

Strong Exponential Time Hypothesis [SETH]

For all $\varepsilon > 0$ there exists a $k \geq 3$ such that kSAT requires $2^{(1-\varepsilon)n}$ time.

Introduction

- Other problems in **P** have similar behaviour:

Introduction

- Other problems in **P** have similar behaviour:
- The Edit Distance problem, for example, admits a classical DP $\mathcal{O}\left(n^2\right)$ algorithm (*Wagner, Fisher 1974*), but till today the best improvement is $\mathcal{O}\left(n^2 / \log^2 n\right)$ (*Masek, Paterson 1980*).

Introduction

- Other problems in **P** have similar behaviour:
- The Edit Distance problem, for example, admits a classical DP $\mathcal{O}\left(n^2\right)$ algorithm (*Wagner, Fisher 1974*), but till today the best improvement is $\mathcal{O}\left(n^2 / \log^2 n\right)$ (*Masek, Paterson 1980*).
- Similar observations hold for the Longest Common Subsequence Problem (LCS).

Introduction

- Other problems in **P** have similar behaviour:
- The Edit Distance problem, for example, admits a classical DP $\mathcal{O}\left(n^2\right)$ algorithm (*Wagner, Fisher 1974*), but till today the best improvement is $\mathcal{O}\left(n^2/\log^2 n\right)$ (*Masek, Paterson 1980*).
- Similar observations hold for the Longest Common Subsequence Problem (LCS).
- The 3SUM problem: *Given a set S of integers, are there $x, y, z \in S$ such that $x + y + z = 0$?* We have a trivial $\mathcal{O}\left(n^2 \log n\right)$ algorithm, and the best known runs in $\mathcal{O}\left(n^2(\log\log n)^2/\log^2 n\right)$ (*Baran, Demaine, Patrascu*, 2008).

Introduction

- The Colinearity Problem in Computational Geometry: *Given n points in the plane, are any three colinear?* The best known algorithms runs in $n^{2-o(1)}$.

Introduction

- The Colinearity Problem in Computational Geometry: *Given n points in the plane, are any three colinear?* The best known algorithms runs in $n^{2-o(1)}$.
- The APSP (All-Pairs Shortest Paths) Problem. There is the classical DP $\mathcal{O}\left(n^3\right)$ algorithm (Floyd, Warshall, 1962). Best known algorithm runs in $\mathcal{O}\left(n^3/exp(\sqrt{\log n})\right)$ (Williams 2014), using Circuit Complexity tools, namely the Razborov-Smolensky polynomials.

Introduction

- The Colinearity Problem in Computational Geometry: *Given n points in the plane, are any three colinear?* The best known algorithms runs in $n^{2-o(1)}$.
- The APSP (All-Pairs Shortest Paths) Problem. There is the classical DP $\mathcal{O}\left(n^3\right)$ algorithm (Floyd, Warshall, 1962). Best known algorithm runs in $\mathcal{O}\left(n^3/exp(\sqrt{\log n})\right)$ (Williams 2014), using Circuit Complexity tools, namely the Razborov-Smolensky polynomials.
- For any of these problems, is there a complexity theoretic reason (a.k.a. barrier) to obtaining a better algorithm?

Introduction

- The Colinearity Problem in Computational Geometry: *Given n points in the plane, are any three colinear?* The best known algorithms runs in $n^{2-o(1)}$.
- The APSP (All-Pairs Shortest Paths) Problem. There is the classical DP $\mathcal{O}\left(n^3\right)$ algorithm (Floyd, Warshall, 1962). Best known algorithm runs in $\mathcal{O}\left(n^3/exp(\sqrt{\log n})\right)$ (Williams 2014), using Circuit Complexity tools, namely the Razborov-Smolensky polynomials.
- For any of these problems, is there a complexity theoretic reason (a.k.a. barrier) to obtaining a better algorithm?
- Or else, **is the reason for this hardness the same for all -or even for some- these problems?**

Introduction

Fine-Grained Approach

- Start with a problem solvable in $\mathcal{O}(t(n))$, and nothing much better known, and formulate a *hypothesis*.
- Connect this problem to others via *fine-grained reductions*, so that for a problem having an $\mathcal{O}(T(n))$ algorithm, obtaining an $\mathcal{O}\left(T^{1-\varepsilon}(n)\right)$ algorithm for $\varepsilon > 0$, would violate the hypothesis.

Introduction

Fine-Grained Approach

- Start with a problem solvable in $\mathcal{O}(t(n))$, and nothing much better known, and formulate a *hypothesis*.
- Connect this problem to others via *fine-grained reductions*, so that for a problem having an $\mathcal{O}(T(n))$ algorithm, obtaining an $\mathcal{O}\left(T^{1-\varepsilon}(n)\right)$ algorithm for $\varepsilon > 0$, would violate the hypothesis.

- **P** vs **NP** is model indepedent, but in a fine-grained analysis, the precise computational model *does* matter.

- We use the *Word RAM model* with $\mathcal{O}(\log n)$ bit words
  (i.e. operations on $\mathcal{O}(\log n)$ bit chunks of data in constant time).

Fine-Grained Standard Conjectures

SETH

For all $\varepsilon > 0$ there exists a $k \geq 3$ such that kSAT requires $2^{(1-\varepsilon)n}$ time.

The 3SUM Conjecture

Any algorithm requires $n^{2-o(1)}$ time to determine whether a set $S \subset \{-n^3, \cdots, n^3\}$ of integers of size $|S| = n$ contains 3 distinct elements $a, b, c \in S$ such that $a + b = c$.

APSP Conjecture

Any algorithm requires $n^{3-o(1)}$ time to compute the distances between every pair of vertices in an n node graph with edge weights in $\{1, \ldots, n^c\}$, for some constant c.

- These conjectures can be extended to randomized algorithms.

ETH, SETH and implications

Exponential Time Hypothesis

- We need a more strict formalization to work with:
- Let:

  $s_k = \inf\{c \mid \text{there is a } \mathcal{O}^*\left(2^{c \cdot n}\right) \text{ algorithm for kSAT with } n \text{ variables}\}$

- Then:

Exponential Time Hypothesis

- We need a more strict formalization to work with:
- Let:

  $s_k = \inf\{c \mid \text{there is a } \mathcal{O}^*\left(2^{c \cdot n}\right) \text{ algorithm for kSAT with } n \text{ variables}\}$

- Then:

ETH
$s_3 > 0$

Exponential Time Hypothesis

- We need a more strict formalization to work with:
- Let:

  $s_k = \inf\{c \mid \text{there is a } \mathcal{O}^*\left(2^{c \cdot n}\right) \text{ algorithm for kSAT with } n \text{ variables}\}$

- Then:

ETH	**SETH**
$s_3 > 0$	$\lim_{k \to \infty} s_k = 1$

The Need for Sparsification

- Can we relate SETH to ETH?
- Recall the classic reduction from **NP**-completeness theory:

Theorem

If 3SAT can be solved in polynomial time, then so can kSAT, for every $k \geq 3$.

- Can we say the same for subexponential time solvability?

The Need for Sparsification

- Can we relate SETH to ETH?
- Recall the classic reduction from **NP**-completeness theory:

Theorem

If 3SAT can be solved in polynomial time, then so can kSAT, for every $k \geq 3$.

- Can we say the same for subexponential time solvability?
- Recall the proof of the above theorem:
  - For every k-clause $C = (x_1 \vee x_2 \vee \cdots \vee x_k)$, we introduce new variables $y_3, y_4, \ldots, y_{k-1}$ and replace C by:

    $$(x_1 \vee x_2 \vee y_3) \wedge (\bar{y}_3 \vee x_3 \vee y_4) \wedge (\bar{y}_4 \vee x_4 \vee y_5) \wedge \cdots \wedge (\bar{y}_{k-1} \vee x_{k-1} \vee x_k)$$

The Need for Sparsification

- Can we relate SETH to ETH?
- Recall the classic reduction from **NP**-completeness theory:

Theorem

If 3SAT can be solved in polynomial time, then so can kSAT, for every $k \geq 3$.

- Can we say the same for subexponential time solvability?
- Recall the proof of the above theorem:
  - For every k-clause $C = (x_1 \vee x_2 \vee \cdots \vee x_k)$, we introduce new variables $y_3, y_4, \ldots, y_{k-1}$ and replace C by:

    $$(x_1 \vee x_2 \vee y_3) \wedge (\bar{y}_3 \vee x_3 \vee y_4) \wedge (\bar{y}_4 \vee x_4 \vee y_5) \wedge \cdots \wedge (\bar{y}_{k-1} \vee x_{k-1} \vee x_k)$$

- If we use the above reduction, we'll **fail**:
  For each clause we introduce $k-3$ variables, thus the new formula has $n + (k-3)m$ variables.
- If we can solve 3SAT in subexponential time, suppose $\mathcal{O}\left(2^{\varepsilon n}\right)$ for some $\varepsilon > 0$, then the above reduction algorithm gives us $\mathcal{O}\left(2^{\varepsilon(n+(k-3)m)}\right)$. If $m = \mathcal{O}\left(n^2\right)$, then it's a disaster.

The Need for Sparsification

- We clearly need an intermediate step here:

Theorem (The Sparsification Lemma)

For all $\varepsilon > 0$ and $k > 0$, there is a constant $C = C(\varepsilon, k)$ such that any kCNF formula ϕ with n variables can be expressed as $\phi = \bigvee_{i=1}^{t} \psi_i$, where $t \leq 2^{\varepsilon n}$ and each ψ_i is a kCNF formula over the same variable set as ϕ and at most Cn clauses.
This disjunction can be computed in $\mathcal{O}^*\left(2^{\varepsilon n}\right)$ time.

- Using the above lemma, we can "sparsify" the kCNF formula, in order to avoid the previously mentioned phenomena.

Exponential Time Hypothesis

Theorem

SETH $\Rightarrow$ ETH

Proof.

- Assume, for the sake of contradiction, that $s_3 = 0$.
- So, for every $\delta > 0$ there exists algorithm A_δ solving 3SAT in $\mathcal{O}^*\left(2^{\delta n}\right)$ time.

Exponential Time Hypothesis

Theorem

SETH $\Rightarrow$ ETH

Proof.

- Assume, for the sake of contradiction, that $s_3 = 0$.
- So, for every $\delta > 0$ there exists algorithm A_δ solving 3SAT in $\mathcal{O}^*\left(2^{\delta n}\right)$ time.
- Consider the following method for solving *kSAT*:
  - Given formula ϕ, apply the sparsification lemma for some $\varepsilon > 0$.
  - We now have in $\mathcal{O}^*\left(2^{\varepsilon n}\right)$ time at most $2^{\varepsilon n}$ ψ_i's, ϕ is satisfiable iff any of the ψ_i's is, and each ψ_i has at most $C(\varepsilon, k)n$ clauses.
  - Now apply the classic reduction from kSAT to 3SAT on any ψ_i (recall that for every k-clause of ψ_i we introduce $k-3$ new variables and $k-2$ new clauses).
  - This results in a 3CNF formula ψ_i' with at most $(1 + kC(\varepsilon, k))n$ variables.

Exponential Time Hypothesis

Theorem

SETH $\Rightarrow$ ETH

Proof. (*cont'd*)

- Now, if we apply the algorithm A_δ to ψ'_i, for some $\delta > 0$, we can solve satisfiability of ψ'_i in $\mathcal{O}^*\left(2^{\delta' n}\right)$, for $\delta' = \delta \cdot (1 + kC(\varepsilon, k))$.
- By applying the procedure to all ψ_i's, we can solve satisfiability of ϕ in $\mathcal{O}^*\left(2^{\delta'' n}\right)$, for $\delta'' = \varepsilon + \delta' = \varepsilon + \delta \cdot (1 + kC(\varepsilon, k))$.
- Since $s_3 = 0$, we can choose ε and δ arbitrarily close to 0, so δ'' is arbitrarily close to 0, hence $s_k = 0$ for $k \geq 3$, which contradicts SETH. □

Fine-Grained Reductions

Fine-Grained Reductions

Fine-Grained Reduction

Let $a(n), b(n)$ be nondecreasing functions of n.
Problem A is (a, b)-reducible to problem B ($A \leq_{a,b} B$), if:
$\forall \varepsilon > 0\ \exists \delta > 0$, an algorithm F with oracle access to B,

- F runs in at most $d \cdot a^{1-\delta}(n)$ time
- F makes at most $k(n)$ oracle queries adaptively
  (the j^{th} instance B_j is a function of $\{B_i, a_i\}_{1 \leq i < j}$)
- The sizes $|B_i| = n_i$ for any choice of oracle answers a_i obey the inequality:

$$\sum_{i=1}^{k(n)} b^{1-\varepsilon}(n_i) \leq d \cdot a^{1-\delta}(n)$$

- Improvements over $b(n)$ for B imply inprovements over $a(n)$ for A.

The Orthogonal Vectors Problem

- A key problem for understanding fine-grained reductions is the Orthogonal Vectors problem.

Orthogonal Vectors (OV)

Let $d = \omega(\log n)$. Given two sets $A, B \subseteq \{0,1\}^d$, with $|A| = |B| = n$, decide whether there exist $a \in A, b \in B$ such that $a \cdot b = 0$.

k-Orthogonal Vectors (kOV)

Let $d = \omega(\log n)$. Given sets $A_1, \ldots, A_k \subseteq \{0,1\}^d$, with $|A_i| = n$ for all $i \in [k]$, decide whether there exist $\alpha_1 \in A_1, \ldots, \alpha_k \in A_k$ such that: $\alpha_1 \cdot \alpha_2 \cdots \alpha_k = \sum_{i=1}^{d} \prod_{j=1}^{k} \alpha_j[\,i\,] = 0$.

- Naïve solutions in:
  - $\mathcal{O}\left(n^k d\right)$ time.
  - $\mathcal{O}\left(2^d n\right)$ time.
- Best known algorithm runs in $\mathcal{O}\left(n^{k-1/\Theta(d/\log n)}\right)$ time [Wil15].

Fine-Grained Reductions

The kOV Hypothesis

There is no (randomized) algorithm that can solve kOV in $n^{k-\varepsilon}\,poly(d)$ time for constant $\varepsilon > 0$.

Fine-Grained Reductions

The kOV Hypothesis

There is no (randomized) algorithm that can solve kOV in $n^{k-\varepsilon}\,poly(d)$ time for constant $\varepsilon > 0$.

- The relation of d to n is crucial to the problem.
- For example, if $d = \frac{1}{2}\log n$, then the second algorithm works in $\mathcal{O}\left(n^{3/2}\right)$.
  - *Moderate* Dimension *OV*: $d = n^{o(1)}$.
  - *Low* Dimension *OV*: $d = \omega(\log n)$.

Fine-Grained Reductions

Theorem (Williams 2005)

SAT $\leq_{2^n, n^k}$ kOV

Fine-Grained Reductions

Theorem (Williams 2005)

SAT $\leq_{2^n, n^k}$ kOV

Proof.

- Let $\psi(n, m)$ be the given formula.
- We can assume, due to the Sparsification Lemma, that ψ has $m = \mathcal{O}(n)$ clauses.
- Split the n variables into k sets $V_1, \ldots, V_k$.
- For every $j \in [k]$, create a set A_j containing a vector $\alpha^j(\phi)$ for each of the $N = 2^{n/k}$ partial t.a.'s. ϕ, where:

$$\alpha^j(\phi)[c] = 0, \text{ iff the } c^{th} \text{ clause of } \psi \text{ is satisfied by } \phi.$$

Fine-Grained Reductions

Theorem (Williams 2005)

SAT $\leq_{2^n, n^k}$ kOV

Proof.

- If for some $\alpha_1(\phi_1), \alpha_2(\phi_2), \ldots, \alpha_k(\phi_k)$:

$$\sum_c \prod_j \alpha_j(\phi_j)[c] = 0$$

  then **for every** c, **there is** some vector $\alpha_j(\phi_j)$ that is 0 in c, so ϕ_j satisfies c.

- Thus, the concatenation $\bigcirc_{\ell=1}^{k} \phi_\ell$ satisfies all clauses.
- Conversely, if ϕ satisfies all clauses, then $\phi_j = \phi \restriction_{V_j}$ and so $\sum_c \prod_j \alpha_j(\phi_j)[c] = 0$.

Theorem (Williams 2005)

SAT $\leq_{2^n, n^k}$ kOV

Proof. (cont'd)

- So, if we can solve kOV in $N^{k-\varepsilon} poly(m)$ time for some $\varepsilon > 0$, then we can solve kSAT in:

$$\mathcal{O}\left((2^{n/k})m + (2^{n/k})^{k-\varepsilon} poly(m)\right) = \mathcal{O}\left(2^{(1-\varepsilon')n}\right)$$

contradicting SETH. □

Fine-Grained Reductions

Theorem (Williams 2005)

SAT $\leq_{2^n, n^k}$ kOV

Proof. (cont'd)

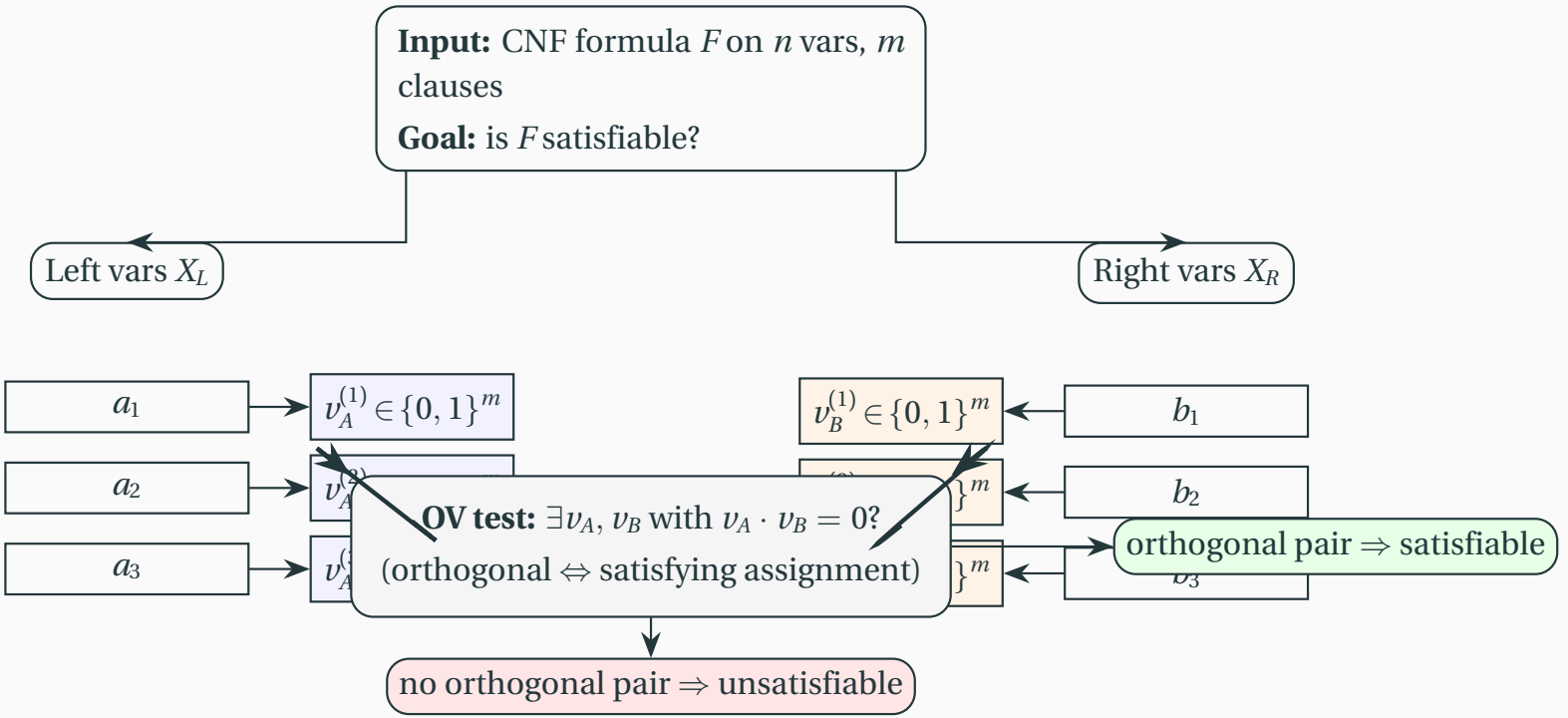
- So, if we can solve kOV in $N^{k-\varepsilon} poly(m)$ time for some $\varepsilon > 0$, then we can solve kSAT in:

$$\mathcal{O}\left((2^{n/k})m + (2^{n/k})^{k-\varepsilon} poly(m)\right) = \mathcal{O}\left(2^{(1-\varepsilon')n}\right)$$

  contradicting SETH. □

- It is simply to see that kOV $\leq_{n^k, n^{k-1}}$ (k − 1)OV $\cdots \leq_{n^3, n^2}$ 2OV.
- So, 2OV is the hardest of these problems.
- The above reduction can be routed through kDOMINATING SET.

Reduction: k-SAT → Orthogonal Vectors



Fine-Grained Reductions

Theorem (Roditty, V. Williams 2013)

If one can distinguish between diameter 2 and 3 in an undirected unweighted graph with $\mathcal{O}(N)$ nodes and edges in $\mathcal{O}\left(N^{2-\varepsilon}\right)$ for some $\varepsilon > 0$, then 2OV on n vectors in d dimensions can be solved in $n^{2-\varepsilon}\, poly(d)$ time and SETH is false.

Fine-Grained Reductions

Theorem (Roditty, V. Williams 2013)

If one can distinguish between diameter 2 and 3 in an undirected unweighted graph with $\mathcal{O}(N)$ nodes and edges in $\mathcal{O}\left(N^{2-\varepsilon}\right)$ for some $\varepsilon > 0$, then 2OV on n vectors in d dimensions can be solved in $n^{2-\varepsilon}\, poly(d)$ time and SETH is false.

Proof.

- Given an instance of 2OV, create a graph G as follows:
- Vertices:
  - For every vector $a \in A$, create a node $a \in V(G)$.
  - For every vector $b \in A$, create a node $b \in V(G)$.
  - For every vector $i \in [d]$, create a node $c_i \in V(G)$.
  - Also, create $x, y \in V(G)$.

Proof. (cont'd)

- Edges:
  - For every $a \in A$ and $i \in [d]$, if $a[i] = 1$, then $(a, c_i) \in E(G)$.
  - For every $b \in B$ and $i \in [d]$, if $b[i] = 1$, then $(b, c_i) \in E(G)$.
  - For every $a \in A$: $(x, a) \in E(G)$, for every $i \in [d]$: $(x, c_i) \in E(G)$.
  - For every $b \in B$: $(y, b) \in E(G)$, for every $i \in [d]$: $(y, c_i) \in E(G)$.
  - $(x, y) \in E(G)$.

Proof. (cont'd)

- Edges:
  - For every $a \in A$ and $i \in [d]$, if $a[i] = 1$, then $(a, c_i) \in E(G)$.
  - For every $b \in B$ and $i \in [d]$, if $b[i] = 1$, then $(b, c_i) \in E(G)$.
  - For every $a \in A$: $(x, a) \in E(G)$, for every $i \in [d]$: $(x, c_i) \in E(G)$.
  - For every $b \in B$: $(y, b) \in E(G)$, for every $i \in [d]$: $(y, c_i) \in E(G)$.
  - $(x, y) \in E(G)$.

- If a, b are **not** orthogonal, then $\exists i : \ a[i] = b[i] = 1$.

- So $d(a, b) = 2$ via the path through c_i.

Proof. (cont'd)

- If a, b are orthogonal, then there is no such c_i and the shortest path from a to b goes through (x, y), and $d(a, b) = 3$.

- All nodes are at distance at most 2 to x, y, c_i, hence the diameter is 3 if there exists an orthogonal pair, and 2 otherwise.

Proof. (cont'd)

- If a, b are orthogonal, then there is no such c_i and the shortest path from a to b goes through (x, y), and $d(a, b) = 3$.

- All nodes are at distance at most 2 to x, y, c_i, hence the diameter is 3 if there exists an orthogonal pair, and 2 otherwise.

- For $N = nd$, $|V(G)|$ and $|E(G)|$ are $\mathcal{O}(N)$, so if this problem can be solved in $\mathcal{O}\left(N^{2-\varepsilon}\right)$ time for some $\varepsilon > 0$, then 2OV is in $\mathcal{O}\left((nd)^{2-\varepsilon}\right) = \mathcal{O}\left(n^{2-\varepsilon} poly(d)\right)$ for some $\varepsilon > 0$. □

Fine-Grained Reductions

- The small universe Hitting Set problem is a variation of the orthogonal vectors problem:

Hitting Set Problem (HS)

Given two sets U, V of n sets each over the universe $[d]$, where $d = \omega(\log n)$, determine whether there is a $u \in U$ that *"hits"* every $v \in V$ in at least one element. That is, **is there** a $u \in U$ such that **for all** $v \in V$ we have:

$$\sum_{i=1}^{d} u_i \cdot v_i > 0$$

Fine-Grained Reductions

- Just as OV, the best known algorith for HS runs in $n^{2-o(1)}$ time. So, it makes sense to define:

Hitting Set Conjecture (HSC)

In the word RAM model with $\mathcal{O}(\log n)$ bit words, any algorithm requires $n^{2-o(1)}$ to determing if U contains a vector that is *not* orthogonal to *any* vector in V.

Fine-Grained Reductions

- Just as OV, the best known algorith for HS runs in $n^{2-o(1)}$ time. So, it makes sense to define:

Hitting Set Conjecture (HSC)

In the word RAM model with $\mathcal{O}(\log n)$ bit words, any algorithm requires $n^{2-o(1)}$ to determing if U contains a vector that is *not* orthogonal to *any* vector in V.

- We will show that HSC $\Rightarrow$ OVC.

Fine-Grained Reductions

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Fine-Grained Reductions

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Proof.

- Let U, V be an HS instance where $|U| = |V| = n$.
- Partition U into s sets $U_1, \ldots, U_s$ of size at most $\lceil n/s \rceil \leq 2n/s$ each, for a parameter s to be set later.

Fine-Grained Reductions

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Proof.

- Let U, V be an HS instance where $|U| = |V| = n$.
- Partition U into s sets $U_1, \ldots, U_s$ of size at most $\lceil n/s \rceil \leq 2n/s$ each, for a parameter s to be set later.
- For every choice of $i, j \in [s]$:
  - While (U_i, V_j) contains an orthogonal pair (u, v), remove $u \in U$ and ask about (U_i, V_j) again.
  - If no orthogonal pair is found, continue to the next choice of (i, j).

Fine-Grained Reductions

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Proof.

- Let U, V be an HS instance where $|U| = |V| = n$.
- Partition U into s sets $U_1, \ldots, U_s$ of size at most $\lceil n/s \rceil \leq 2n/s$ each, for a parameter s to be set later.
- For every choice of $i, j \in [s]$:
  - While (U_i, V_j) contains an orthogonal pair (u, v), remove $u \in U$ and ask about (U_i, V_j) again.
  - If no orthogonal pair is found, continue to the next choice of (i, j).
- At the end, if U contains some u, it must be **non-orthogonal** to all vectors in V, hence the HS instance is a "yes" instance.
- Othewise, if U is empty, then every $u \in U$ is orthogonal to some $v \in V$, and the HS instance is a "no" instance.

Fine-Grained Reductions

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Proof. (cont'd)

- Every call to the OV oracle, either:
  - returns an orthogonal pair (u, v)
  - determines that no such pair exists in $U_i \times V_j$.

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Proof. (cont'd)

- Every call to the OV oracle, either:
  - returns an orthogonal pair (u, v)
  - determines that no such pair exists in $U_i \times V_j$.
- The number of times an orthogonal pair can be returned is at most n.
- Otherwise, the number of calls to OV that return a "no" are at most s^2.

Fine-Grained Reductions

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Proof. (cont'd)

- Every call to the OV oracle, either:
  - returns an orthogonal pair (u, v)
  - determines that no such pair exists in $U_i \times V_j$.
- The number of times an orthogonal pair can be returned is at most n.
- Otherwise, the number of calls to OV that return a "no" are at most s^2.
- Thus, if we set $s = \sqrt{n}$, the number of OV instances created is at most $2n$ and their sizes are at most $2\sqrt{n}$.

Theorem ()

HS $\leq_{n^2, n^2}$ 2OV

Proof. (cont'd)

- Hence, for every $\varepsilon > 0$, there is a fine-grained reduction, creating at most $2n$ instances of size at most $2\sqrt{n}$, and:

$$\sum_{i=1}^{2n} n_i^{2-\varepsilon} \leq 4 \sum_{i=1}^{2n} n^{1-\varepsilon/2} = 8n^{2-\varepsilon/2}$$

- The reduction does simple partitioning of the instance, and removals from U, hence it runs in $\mathcal{O}\left(n^{2-\varepsilon/2}\right)$. □

Thank You!